This is an impressive project that demonstrates a high level of full-stack technical competency, particularly in how you've bridged the gap between raw data APIs and a sophisticated AI user experience.

Here is a full breakdown of the application, structured for your website to highlight your skills while maintaining a professional, "developer-to-developer" tone.

Project Spotlight: "Dreamy" – YouTube Analytics Intelligence Platform

The Vision

Data is only as good as its delivery. For "Dreamy," I built a specialized analytics suite for a YouTube sleep-aid channel. The goal was to move beyond static dashboards and create an interactive consultant that doesn't just show numbers but interprets them within the unique context of the relaxation niche (where traditional metrics like "high click-through" often matter less than "uninterrupted viewer experience").

How It Works: The Architecture

1. Data Extraction & Pipeline (`YouTube APIs + Pandas`)

The engine starts by authenticating via OAuth2 to pull from two distinct sources: the **YouTube Data API (v3)** for video metadata and the **YouTube Analytics API (v2)** for deep-performance metrics.

- **Complex Aggregation:** I developed a custom analysis pipeline using **Pandas** to classify content into "Shorts" vs "Long-form" (based on duration and aspect ratio) and calculate niche-specific metrics like "Watch Time per View" and "Subscriber-to-View Velocity."
- **Temporal Normalization:** The system handles time-zone conversions (UTC to PST) to align posting times with actual viewer peak-activity hours, providing actionable insights into when a global audience is most likely to "drift off."

2. Automated Reporting & Notification Engine

I implemented a dual-reporting cadence to balance high-level strategy with tactical updates:

- **The Bi-Weekly Strategy:** A comprehensive, multi-page HTML report generated via Python that visualizes geographic distribution, device types (crucial for sleep content, where TV views dominate), and follower growth patterns.
- **The 3-Day Pulse:** A lightweight AI-driven analysis triggered every 72 hours. When the analysis is ready, a script automatically generates an update and triggers an **SMTP email notification** to the creator, ensuring they stay informed without needing to check a dashboard.

3. The AI "Consultant": Personable Hybrid RAG

At the heart of the project is "Dreamy," a chatbot with a distinct personality—fun, slightly sarcastic, and "honestly" helpful.

- **Hybrid RAG Architecture:** To allow Dreamy to answer follow-up questions with 100% accuracy, I built a custom Retrieval-Augmented Generation system using a hybrid approach:
  - **TF-IDF & Cosine Similarity:** For semantic relevance across the video dataset.
  - **BM25 (Okapi):** For precise keyword matching on specific video titles or tags.
  - **Metadata Filtering:** The bot can filter its own knowledge base (e.g., "How do my *Friday* videos compare?") before processing the query, significantly reducing LLM "hallucinations."
- **Advanced Prompt Engineering:** I utilized complex, multi-variable system prompts (XML-tagged for structure) that enforce Dreamy's persona while requiring it to perform a multi-step "Information Processing" phase before responding.

4. Deployment & Lifecycle Management

- **Hosting:** The application is hosted on **PythonAnywhere**, where I manage server-side sessions, file-based persistence for chat histories (JSON), and scheduled task execution for the reporting cycles.
- **State Management:** I implemented a robust caching and serialization system using `pickle` to store executive plans and datasets, ensuring the app remains responsive even when handling large volumes of YouTube data.

Key Skills Demonstrated

- **API Mastery:** Deep integration with Google/YouTube OAuth2 and Analytics endpoints, as well as the Anthropic (Claude) API.
- **Data Science:** Designing the logic for content classification and engagement-ratio analysis.
- **Vector Search & NLP:** Building a Hybrid RAG from scratch using `scikit-learn` and `rank_bm25` rather than relying on black-box external services.
- **Full-Stack Python:** Using Flask for the web layer, SMTP for notifications, and Pandas for the heavy lifting of data analysis.
- **Persona Design:** Creating a "Thang" (Dreamy) that feels human and engaging, moving AI beyond a sterile chat interface.

A Humble Developer Note

> "Building Dreamy was a lesson in edge-case management. Handling the nuances of the YouTube API—like privacy thresholds on demographic data or the specific ISO 8601 duration formats—required a lot of debugging and custom parsing. It was a rewarding challenge to take that technical complexity and wrap it in a sarcastic, lofi-loving chatbot that actually makes data fun to look at."